

Mandatory Assignment 3 - Inventory Management System



LEARNING OBJECTIVES

- Working with Python dictionaries and lists
- Implementing a menu-based Python program that is close to a real-world use case.



The most typical mistakes that students do:

- They do not read the instructions carefully. Please see how you will be graded section
 - especially how the sorted is printed based on even numbers and odd numbers
- they do not read the coding rules
- they do the assignment without reading about the theory
- they have not seen the assignment overview video on Canvas



Prerequisites

Everything upto Module 4



Task

Create an **Inventory Management System** that simulates managing a store or warehouse inventory. The system allows the user to track items, update quantities, calculate values, and perform data operations. This project integrates all concepts you have learned in this course

Features

1. Add a New Item to Inventory

- Allow the user to add an item with the following attributes:
 - Item Name (String)
 - Item ID (Unique Integer)
 - Quantity (Integer)
 - Price Per Unit (Float)

2. Update Item Details

- Allow the user to update an item's name, quantity, or price.

3. Remove an Item

- Remove an item from the inventory using its ID.

4. View Inventory

- Display all items with details such as:
 - Name, ID, Quantity, Price, and Total Value (Quantity × Price).

5. Check Inventory Value

- Calculate the total value of all items in the inventory.

6. Find Items with Low Stock

- Identify and list items with quantities below a user-defined threshold.

7. Exit Program

- Exit the program when the user chooses.

Requirements

- Use loops (`for` , `while`) to navigate the menu and iterate over inventory data.
- Use functions to encapsulate actions like adding items, updating items, and calculating values.
- Use lists and dictionaries to store inventory data.
- Implement error handling to ensure valid input (e.g., numeric quantities and prices).
- Include basic arithmetic operations for inventory value calculations.

Code Outline

1. Data Storage

Use a dictionary where keys are item IDs and values are dictionaries containing the item's name, quantity, and price as shown below:

```
inventory = {  
    1001: {"name": "Laptop", "quantity": 10, "price": 800.50},  
    1002: {"name": "Monitor", "quantity": 5, "price": 200.0},  
}
```

2. Functions

Create functions for each feature:

- `add_item()`
- `update_item()`
- `remove_item()`
- `view_inventory()`
- `check_total_value()`
- `find_low_stock()`

3. Menu and Loop

Use a `while` loop to display the menu and call the appropriate function based on the user's choice.

4. Error Handling

Ensure valid input for numeric fields (e.g., quantity, price) and unique IDs.

Expected Output (by menu item)

Use these examples to verify your program's interaction and messages. Wording, capitalization, and spacing matter.

1. Add a new item

```
Adding a New Item
Enter item ID (unique integer): 101
Enter item name: Pencil
Enter quantity: 50
Enter price per unit: 2.5
Item 'Pencil' added successfully!
```

ID already exists:

```
Adding a New Item
Enter item ID (unique integer): 101
Error: Item ID already exists. Try a different ID.
```

Invalid numeric inputs:

```
Enter item ID (unique integer): abc
Invalid input! Please enter a valid integer for the item ID.
```

```
Enter quantity: -5
Quantity cannot be negative.
```

```
Enter price per unit: hello
Invalid input! Please enter a valid number for the price.
```

2. Update item details

```
Updating Item Details
Enter item ID to update: 101
Enter new values (press Enter to keep existing values):
Current Name: Pencil | New Name: Pen
Current Quantity: 50 | New Quantity: 100
Current Price: 2.5 | New Price: 3.0
Item updated successfully!
```

Keep existing values:

```
Updating Item Details
Enter item ID to update: 101
Enter new values (press Enter to keep existing values):
Current Name: Pen | New Name:
Current Quantity: 100 | New Quantity:
Current Price: 3.0 | New Price:
Item updated successfully!
```

Item not found:

```
Updating Item Details
Enter item ID to update: 999
Error: Item not found.
```

Invalid numbers:

```
Current Quantity: 100 | New Quantity: abc
Invalid input! Enter a valid integer for quantity.
```

3. Remove an item

```
Removing an Item
Enter the ID of the item to remove: 101
Item removed successfully!
```

Item not found:

```
Removing an Item
Enter the ID of the item to remove: 888
Error: Item not found.
```

4. View inventory

With items:

```
Current Inventory:
-----
ID      Name      Quantity  Price    Total Value
-----
101     Pen          100      3.00     300.00
102     Notebook     25      12.50     312.50
-----
```

Empty:

```
Current Inventory:
No items in inventory.
```

5. Check total inventory value

```
Total inventory value: 612.50
```

6. Find items with low stock

Some items below threshold:

```
Checking for Low Stock Items
Enter the low stock threshold: 50
```

```
Items with Low Stock:
- Name: Notebook, ID: 102, Quantity: 25
```

No items below threshold:

```
Checking for Low Stock Items
Enter the low stock threshold: 10
No items with low stock.
```

Invalid threshold input:

```
Enter the low stock threshold: hello
Invalid input! Please enter a valid integer.
```

7. Exit

```
Goodbye!
```

Invalid menu option

```
Invalid option! Please choose a valid number.
```



Tips

Formatted printing for menu item 3

The easiest way is to use **f-strings** and specify how wide each column should be. For example:

```
print(f"{'ID':<6}{'Name':<15}{'Quantity':>10}{'Price':>10}{'Total Value':>15}")
print("-" * 60)
print(f"{'101':<6}{'Pen':<15}{'100':>10}{3.00:>10.2f}{300.00:>15.2f}")
```

A few notes:

- `:<6` means *left-align* within 6 spaces (great for text).
- `:>10` means *right-align* within 10 spaces (great for numbers).
- Use `^` for center align
- `.2f` ensures prices and totals always show **two decimal places**.

- Use `print("-" * 60)` to add nice separator lines.

You might see examples using the older `.format()` method, but f-strings are the modern and preferred way, cleaner and easier to read (According to me).

Try adjusting the numbers (6, 15, 10, etc.) until your columns look balanced. You'll get a nicely formatted inventory table.



How will you be graded / What do you need to take care of

- You will be graded and checked for functional and non-functional requirements
- Functional requirements
 - The functions are exactly as described; the names should match
 - They need to perform what they're supposed to as mentioned in the task description
- Non-functional requirements
 - Strictly follow the coding rules
 - Use only the constructs and knowledge you have learnt in this course.

Stuck? Need a Guide?

You should try this first by yourself. If you are stuck or need more help. Then use the [Guide for Mandatory Assignment 3 - Inventory Management System](https://canvas.ltu.se/courses/24507/pages/guide-for-mandatory-assignment-3-inventory-management-system) (<https://canvas.ltu.se/courses/24507/pages/guide-for-mandatory-assignment-3-inventory-management-system>)

Getting help

Seems like many have not been reading my announcements. I have specifically requested how to seek help and what information to send me when you email or post on Discussions. I have also mentioned that it is not allowed to post answers to the Mandatory Assignments. If you need help, please see [How to get help the right way](https://canvas.ltu.se/courses/24507/pages/how-to-get-help-the-right-way) (<https://canvas.ltu.se/courses/24507/pages/how-to-get-help-the-right-way>)